

Decoupling Vector Data and Index Storage for Space Efficiency

Yuanming Ren¹, Juncheng Zhang¹, Yanjing Ren¹, Rui Yang², Di Wu², and Patrick P. C. Lee¹

¹The Chinese University of Hong Kong ²Bytedance

Abstract

Managing large-scale vector datasets with disk-resident graph approximate nearest neighbor search (ANNS) systems incurs substantial storage overhead due to the co-location of vector data and auxiliary index metadata, which prevents the storage layer from exploiting their distinct compressibility. We present DecoupleVS, a component-aware compressed storage framework for disk-resident graph vector search. Leveraging data-index decoupling as a foundation, DecoupleVS losslessly compresses each component according to its distinct compressibility characteristics, thereby significantly reducing storage space. It further adapts the search and update paths to preserve their performance under compressed storage layouts. Evaluation on real-world public and proprietary billion-scale datasets shows that DecoupleVS reduces storage space by up to 58.7%, while delivering improved or competitive search and update performance compared to state-of-the-art disk-resident graph ANNS systems.

1 Introduction

The rapid growth of AI applications, including web search [12], recommendation [5], and retrieval-augmented generation [29], has spurred demand for managing increasingly large collections of high-dimensional *vectors*. These vectors encode unstructured data (e.g., text, images, audio, video) in a common embedding space, making similarity search a core primitive in modern data-intensive systems. For example, graph-based approximate nearest neighbor search (ANNS) organizes vectors as vertices and connects similar vectors with edges, and improves search efficiency by returning an approximate top- K result set instead of exact neighbors, trading a small accuracy loss for large performance gains.

A vector dataset typically comprises both *vector data* (the data component), full-precision vectors that store embedding values for accurate distance computation, and an *auxiliary index* (the metadata component), which stores per-vector metadata for efficient ANNS. As vector datasets scale to hundreds of trillions of vectors [15], in-memory ANNS systems (e.g., HNSW [23]) become impractical due to prohibitive memory requirements [47]. This motivates *disk-resident* graph ANNS systems (e.g., [2, 17, 36, 44]) that offload vector datasets to solid-state drives (SSDs) or other persistent storage media, thereby enabling billion-scale deployments.

Existing disk-resident graph ANNS systems are mainly optimized for search efficiency by co-locating vector data and

auxiliary index metadata in storage. This simplifies implementation and amortizes I/Os across vector data and metadata access in a single read. However, this search-friendly layout is *storage-inefficient*. First, the total size of graph metadata and vector data can be larger than the raw data (§2.2). Second, fixed-size page-aligned records can suffer from internal fragmentation when the combined vector data and metadata size does not align with the storage page size. Third, co-locating vector data and metadata forces the storage layer to treat semantically distinct components as a single opaque record, preventing tailored compression for each component.

In this paper, we study the storage-compression problem in disk-resident graph ANNS systems. Our insight is that vector data and auxiliary index metadata exhibit fundamentally distinct characteristics in data semantics and access patterns (§2.3), enabling *component-aware compression* to achieve substantially higher compression ratios than general-purpose alternatives. A natural way to exploit these differences is to separate vector data from auxiliary index metadata, allowing each to be compressed, laid out, and accessed independently. While data-index decoupling has also been considered concurrently by prior work [22, 48] (§5), applying this idea to storage compression introduces three non-trivial challenges. First, general-purpose lossless compression techniques (e.g., LZ4 [3], Huffman [19]) should be adopted for vector storage to preserve data fidelity, yet they fail to exploit the semantic structure of vector data and auxiliary index metadata; improper use of them yields suboptimal compression gains. Second, compression produces variable-size data that cannot be directly addressed by vector ID, requiring additional metadata management to support efficient random access. Third, separating vector data and auxiliary index metadata into independent physical storage introduces I/O locality loss during search and consistency challenges during updates.

We design DecoupleVS, a decoupled vector storage management framework for disk-resident graph ANNS systems. Using data-index decoupling as an enabling substrate, DecoupleVS adopts a co-design of lossless compression, data layouts, and search and update paths without compromising performance. Instead of proposing new compression primitives, DecoupleVS shows how compression can be made practical for compressed decoupled storage and random-access graph ANNS workloads through: (i) component-aware lossless encoding of vector data and auxiliary index metadata, (ii) hierarchical layouts that enable direct access over variable-size compressed records, (iii) latency-aware search that re-

moves vector data reads from the traversal critical path, and (iv) decoupled update paths that combine graph batch merges with log-structured vector data storage.

We conduct experiments on real-world public and proprietary datasets. Compared to state-of-the-art disk-resident graph ANNS systems (DiskANN [17] and PipeANN [13]), DecoupleVS saves up to 58.7% of storage space, achieves up to 2.39 $\times$ search throughput gain, and preserves high update efficiency. We will open-source DecoupleVS in the final paper.

2 Background

2.1 Basics of ANNS

An ANNS system identifies the most similar items in a large vector dataset for a search query. It employs deep-learning-based embedding models to map unstructured data into high-dimensional vectors within a common vector space, and constructs an auxiliary index over these vectors for efficient similarity search. Given a search query, the system generates a query vector and uses the auxiliary index to locate the most similar vectors according to a distance metric (e.g., Euclidean distance, inner product, or cosine similarity), then returns the top- K items corresponding to the nearest vectors. Each vector typically comprises hundreds of dimensions encoded as numerical data types, such as UINT8, INT8, or FP32 [27], where each dimension captures a specific semantic feature learned by the embedding model [31]. To maintain search accuracy, vectors are often normalized to prevent large-value dimensions from dominating distance calculations. Vector dimensionality can reach tens of thousands when representing rich textual data (e.g., in large language models) [1].

The auxiliary index enables efficient search without scanning the entire dataset, and can be structured as graph-based [17, 23], cluster-based [2, 18], or tree-based [26]. For example, a graph-based index represents each vector as a vertex and connects similar vertices by edges. Given a search query, the system begins at a designated entry point, traverses the graph along edges, and computes distances between the query vector and visited vertices to construct a candidate set. If no closer vectors can be found, it returns the top- K candidates as the *result set*. In-memory ANNS systems (e.g., HNSW [23], IVFADC [18], and RUMMY [47]) store both vectors and auxiliary indexes in DRAM for fast access, but incur substantial memory overhead (e.g., RUMMY [47] requires terabytes of memory to manage billion-scale datasets), motivating the adoption of disk-resident ANNS systems.

2.2 Disk-Resident ANNS Systems

To reduce memory footprints, disk-resident ANNS systems (e.g., [2, 17, 36, 44]) adopt a hybrid storage architecture that retains compact in-memory representations while persisting full-precision vector data and auxiliary index metadata on disk. For example, DiskANN [17] keeps lossy-compressed

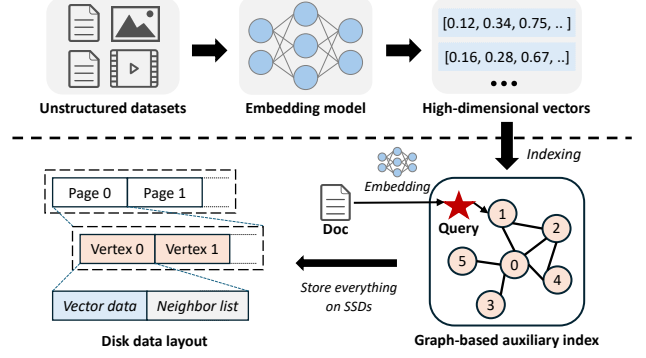


Figure 1. A disk-resident graph ANNS system based on DiskANN.

vectors in memory (via product quantization (PQ) [18]), while SPANN [2] keeps an in-memory graph of cluster centroids, each linked to an on-disk posting list of full-precision vector data. However, SPANN replicates each vector up to eight times across different posting lists to maintain high search accuracy, incurring substantial storage overhead (§4.3). Given that DiskANN exhibits a more space-efficient design, we focus on it for problem motivation.

Figure 1 depicts the storage layout of a disk-resident graph ANNS system based on DiskANN, with graph index vertices sorted by ID and stored sequentially. Each vertex bundles a full-precision vector with its neighbor list, enabling both to be fetched in a single disk I/O during graph traversal. Since vertices are fixed-size and page-aligned, DiskANN can directly compute any vertex’s disk offset from its ID without additional metadata lookups. For a search query, DiskANN maintains a candidate list of size L , initialized with the neighbors of a designated entry point ranked by their PQ-based distances to the query vector. It then expands the search frontier iteratively: in each iteration, it issues batch I/O requests to fetch the top- W unvisited candidates (W is the *beam width*) and evaluates their neighbors via in-memory PQ codes to select the most promising candidates without reading full-precision vectors from disk. Finally, it re-ranks the visited vertices using their full-precision vectors to return the top- K result set. Despite various I/O optimizations, existing disk-resident ANNS systems still face two critical limitations at the persistent storage layer.

Limitation #1: High storage costs. Disk-resident graph ANNS systems incur huge storage overhead for persisting both high-dimensional vector data and auxiliary index metadata. For instance, a 128-dimensional vector [20] with each dimension encoded as a 4-byte float consumes 512 bytes. The auxiliary index connecting each vector to 128 neighbors [13] adds another 512 bytes of 4-byte integer neighbor IDs, bringing the total storage per vector to 1 KiB (i.e., doubling the raw vector size). This overhead compounds severely at scale: vector datasets can grow to hundreds of trillions of vectors in production [15], and the footprint is further amplified by replication for fault tolerance and load balancing [42].

Limitation #2: High I/O demands. Disk-resident graph

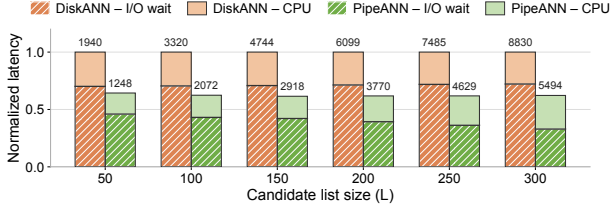


Figure 2. Search latency breakdown of DiskANN and PipeANN (CPU time vs. I/O wait time), normalized to DiskANN’s latency. Numbers above bars are absolute latencies (μs).

ANNS systems suffer from significant I/O amplification in both search and update operations. Search queries must traverse large portions of the auxiliary index on disk to identify the result set. This disk-bound traversal can account for over 70% of the total query time [21], and this bottleneck persists even with pipelined I/O and computation [13]. To verify, we profile DiskANN [17] and PipeANN [13] on the SIFT1M dataset with 1 M 128-dimensional vectors¹ [20]. We measure the *I/O wait time* (i.e., the duration that CPU threads are idle awaiting disk I/Os) with beam width $W = 4$ (as recommended in [17]) while varying the candidate list size L from 50 to 300 to cover various search accuracy levels. Figure 2 shows that even with pipelining, PipeANN’s I/O wait time still accounts for 52.9–72.2% of overall search latency, confirming that the I/O demands of graph traversal cannot be fully alleviated by pipelining alone.

Updates are also costly: each vector update modifies not only vector data but also all relevant auxiliary index metadata. For example, for a graph-based index, updating a vector also revises the adjacency lists of all its neighbors. Even with buffered updates [34], the auxiliary index still needs to be periodically rewritten to disk in full to incorporate accumulated changes, consuming substantial write bandwidth.

2.3 Motivation and Challenges

Vector data and auxiliary index metadata have distinct characteristics. We identify two key differences that lead to storage and I/O inefficiencies under co-located storage.

(i) *Different data semantics.* Vector data generated by embedding models encodes high-dimensional numerical features with dimension-wise statistical regularity, while auxiliary index metadata primarily contains neighbor IDs as fixed-length integers (e.g., adjacency lists in graph-based indexes [17] or posting lists in a cluster-based index [2]). Co-locating them in storage forfeits opportunities for component-aware optimizations on each.

(ii) *Different access patterns.* During search, the auxiliary index is traversed frequently to identify candidate vectors, while vector data is accessed only sporadically for final re-ranking [17] (using full-precision vectors for accurate distance computation). During updates, auxiliary index

¹We choose this small-scale dataset for ease of analysis and visualization, yet our findings hold consistently for billion-scale datasets (§4).

metadata must be kept current to maintain search accuracy [43, 44], while vector data can be updated asynchronously without compromising correctness. Such distinct access patterns motivate independent I/O handling for them.

Design considerations. To address the above storage and I/O inefficiencies, we explore decoupled storage management for vector data and auxiliary index metadata. This raises four design questions.

Q1: How to maximize storage savings? Lossless compression is necessary to preserve data integrity: full-precision vectors are required for accurate final re-ranking [17], and the auxiliary index must remain exactly correct for search algorithms. However, general-purpose lossless techniques, such as entropy coders (e.g., Huffman [19], ANS [6]) and dictionary coders (e.g., LZ77 [32], LZ4 [3]), treat data as opaque byte streams and fail to exploit the semantic structure of vectors or the integer-sequence regularity of auxiliary index metadata, yielding suboptimal compression ratios.

Q2: How to mitigate the operational overhead on compressed data? Compression introduces two forms of overhead. First, decompression at query time adds computational costs that must not degrade search or update performance. Second, compression produces variable-size outputs, eliminating the fixed-stride layout that allows direct disk addressing by vector ID. Variable-size data requires additional metadata management and indirection to support efficient random access.

Q3: How to maintain high search performance and accuracy? Separating vector data and auxiliary index metadata in physical storage eliminates I/O locality, forcing multiple I/Os to distinct storage locations per graph traversal step. This extra I/O overhead risks degrading search performance; any mitigation strategy must preserve recall accuracy to the same level as co-located designs.

Q4: How to support updates under a decoupled storage layout? Ensuring consistency across decoupled components requires coordinated updates to both vector data and auxiliary index metadata on disk [14, 34]. While decoupling breaks spatial locality, it also creates an opportunity to handle updates for each component independently, potentially reducing overall write amplification.

3 DecoupleVS Design

3.1 Design Overview

DecoupleVS is a compressed storage framework for disk-resident graph ANNS systems. Its primary objective is to reduce the persistent storage footprint of vector data and auxiliary index metadata while preserving the random-access interface for existing graph search and update algorithms. Figure 3 shows DecoupleVS’s architecture, which builds on DiskANN [17]. DecoupleVS stores the auxiliary index (including each vertex’s neighbor count and neighbor list) and vector data in separate on-disk files; this decoupled organization is not the main contribution by itself, but rather enables

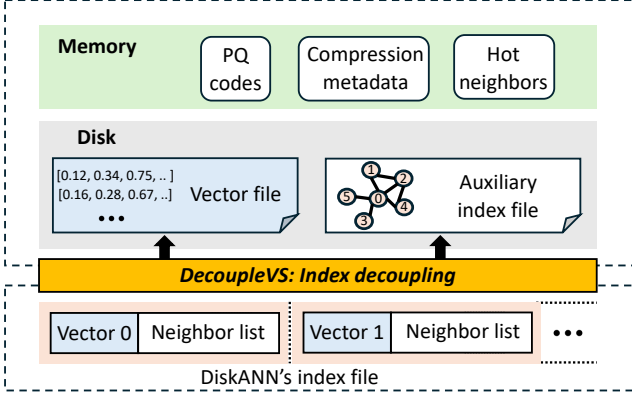


Figure 3. Architecture of DecoupleVS.

DecoupleVS to apply component-aware compression. Like DiskANN, DecoupleVS caches PQ-compressed vectors in memory; additionally, it maintains compression metadata and hot neighbor lists in memory to support efficient compressed storage access (§3.4). DecoupleVS introduces four techniques to optimize space efficiency, search, and updates.

- **Tailored lossless compression** (§3.2). DecoupleVS applies component-aware lossless compression schemes to vector data and auxiliary index metadata, exploiting their distinct compressibility characteristics.
- **Hierarchical storage layouts** (§3.3). DecoupleVS organizes compressed vector data and auxiliary index metadata in hierarchical on-disk layouts that enable efficient random access and updates while bounding the in-memory compression metadata footprint.
- **Latency-aware vector search** (§3.4). DecoupleVS assigns different I/O paths to vector data and auxiliary index metadata based on their distinct roles in search, keeping index access on the critical path while offloading vector fetching. It also uses adaptive prefetching to avoid unnecessary I/O.
- **Decoupled vector updates** (§3.5). DecoupleVS separates the update paths for vector data and auxiliary index metadata. It batch-merges index updates to preserve graph integrity [34], while appending vector data in a log-structured manner and deferring space reclamation to background garbage collection, achieving low write amplification.

Applicability. DecoupleVS is built and evaluated for disk-resident graph ANNS systems derived from DiskANN. It demonstrates how separating vector data from auxiliary index metadata enables component-aware compression and access optimizations, and we expect its core design principles to extend beyond the current implementation. In particular, vector data compression is independent of the graph structure and applies to systems that store full-precision vectors on disk, while auxiliary index metadata compression exploits a common property that both graph-based (neighbor lists) and cluster-based (posting lists) systems index vectors by integer identifiers that can often be reordered into monotonically increasing sequences for compact encoding.

Table 1. Characterization of different datasets.

Metric	SIFT1M	SPACEV1M	DecoupleVS1M
Global dispersion	36.2	12.2	0.09
Dimensional dispersion	32.8	9.69	0.06
Global entropy	2.63	5.59	4.39
Columnar entropy	1.73	5.46	2.86

Nevertheless, cluster-based systems (e.g., SPANN) have different access patterns and posting-list layouts. Extending DecoupleVS’s design to those systems requires additional engineering and is left for future work.

3.2 Tailored Lossless Compression

Vector data characterization. We characterize the compressibility of vector data and auxiliary index metadata using three 1 M-vector datasets: (i) SIFT1M [20] with 128 dimensions, (ii) SPACEV1M [24] with 100 dimensions, and (iii) DecoupleVS1M, a 128-dimension proprietary dataset from a commercial application (billion-scale validation is deferred to §4). Since vectors are normalized (§2.1), we hypothesize that values within the same dimension exhibit lower variation than the global distribution. We measure this via: (i) *global dispersion*, the standard deviation across all values in the dataset, and (ii) *dimensional dispersion*, the average per-dimension standard deviation. Table 1 confirms that dimensional dispersion is consistently lower than global dispersion across all three datasets, indicating that values within each dimension are more concentrated and hence more compressible. We further quantify compressibility via information entropy [33] based on: (i) *global entropy*, computed across all bytes in the dataset, and (ii) *columnar entropy*, the average entropy of each byte column across all vectors. Table 1 shows that columnar entropy is consistently lower than global entropy across all datasets, by 34.3% and 34.8% for SIFT1M and DecoupleVS1M, respectively. This indicates strong byte-positional locality that is exploitable for compression.

Delta compression for multi-dimensional vectors. Our entropy analysis shows that compressing values at the same byte position across vectors yields high savings. However, doing so directly requires a separate compression stream per byte position, causing numerous decompression operations that severely degrade retrieval performance. Instead, DecoupleVS constructs a base vector by choosing the most frequent byte value at each byte position across all vectors being considered. It then applies XOR-based delta compression [16, 41]: each vector is bitwise-XORed with the base vector to produce a low-entropy XOR-delta. The XOR-deltas are then losslessly compressed using Huffman coding [19], which offers fast decoding and near-optimal compression ratios [45]. This preserves byte-positional locality while enabling vector-level compression through a single, unified compression stream. Compared to prior delta compression approaches [16, 28, 41], which operate on one-dimensional byte sequences, DecoupleVS explicitly handles multi-dimensional vectors by con-

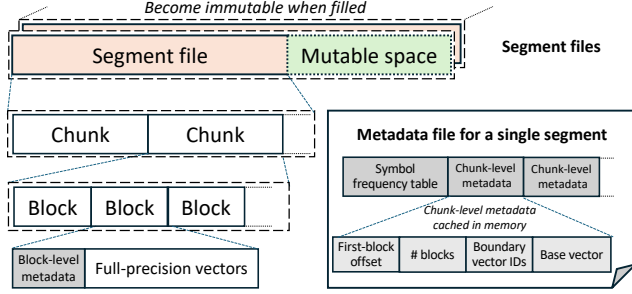


Figure 4. Hierarchical storage layout for vector data.

structuring a dimension-aligned base vector.

Auxiliary index compression. Each vector’s auxiliary index metadata comprises a neighbor count and an adjacency list of integer IDs. Since all neighbors are evaluated during search in an order-independent manner, DecoupleVS sorts neighbor IDs in ascending order and transforms each adjacency list into a monotonically increasing integer sequence, making it highly compressible. It then applies the *Elias-Fano* algorithm [7, 8] to encode the sorted list using a *two-level representation*: the lower bits of each ID are stored at full precision with a fixed width across all IDs, while the higher bits of all IDs are encoded collectively in a compact bitmap.

3.3 Hierarchical Storage Layouts

DecoupleVS introduces hierarchical storage layouts for vector data and auxiliary index metadata to enable efficient random access and updates to variable-size compressed data. **Segment-level vector data organization.** As shown in Figure 4, DecoupleVS organizes vector data into fixed-capacity *segment* files to bound the maximum number of vectors being stored. A segment is initially mutable and accepts new vectors via log-structured writes. Once a segment reaches capacity, it is sealed and made immutable; a background thread pool then compresses it in parallel. Segment IDs start from 0, and a given vector ID is mapped to its segment by integer division of the vector ID by the segment capacity. During updates, DecoupleVS asynchronously frees invalid vectors once the stale data volume in a segment exceeds a predefined threshold (§3.5), reclaiming space without rewriting the entire index as in prior work [34].

Each compressed segment is subdivided into fixed-capacity *chunks*. In each chunk, vectors are packed into 4 KiB physical disk blocks, the minimum I/O units, and sorted by IDs. To locate the block containing a given vector ID, DecoupleVS maintains in-memory chunk-level metadata, including the offset of the chunk’s first block (4 bytes), the number of blocks (4 bytes), the boundary vector IDs of all blocks (4 bytes each), and a base vector for delta compression (§3.2). All chunk-level metadata for a segment is persisted to a separate disk file that is loaded into memory at startup for fast lookups without scanning the segment. To retrieve a vector, DecoupleVS queries the in-memory chunk metadata to locate the target block, issues a single disk read for the

block, and decompresses the target vector. This three-level hierarchy (segment→chunk→block) bounds the metadata memory footprint while preserving efficient random access. **Segment-level vector compression.** DecoupleVS’s Huffman-based compression (§3.2) requires a symbol frequency table. A single global table over the entire dataset ignores local data statistics, while per-chunk tables incur excessive metadata overhead. Thus, DecoupleVS builds one table per segment, balancing compression fidelity and metadata costs. It performs compression in two stages. In the first stage, it determines whether to apply XOR-based delta compression (§3.2) to each chunk. It samples a portion of the vectors (e.g., the first 10%) and computes the entropy of both the raw vectors and their XOR-deltas against a candidate base vector, built by selecting the most frequent byte value at each byte position within the chunk, and applies delta compression to the whole chunk only if it yields lower entropy. This ensures that delta compression is applied only when it achieves storage savings. DecoupleVS stores the outputs, along with the base vector (if used), in the chunk-level metadata. In the second stage, DecoupleVS constructs a unified symbol frequency table from all chunks in the segment and encodes every vector using the resulting Huffman code. The frequency table is persisted alongside the chunk-level metadata (Figure 4), and is reconstructed in memory at segment load time and cached for fast decompression during search.

Block-based auxiliary index organization. DecoupleVS stores the compressed auxiliary index in a block-based layout analogous to that of vector data: each 4 KiB disk block holds multiple compressed adjacency lists preceded by a block-level metadata header. To enable fast random access, DecoupleVS maintains a sparse in-memory index mapping boundary vector IDs to block offsets, so that any target adjacency list can be located with a single lookup. This sparse index is persisted to disk and loaded into memory at startup.

Compression metadata memory management. DecoupleVS keeps three types of compression metadata in memory: (i) chunk-level metadata, (ii) per-segment frequency tables, and (iii) the sparse auxiliary index. The chunk-level metadata footprint depends on chunk size C , creating a trade-off between memory usage and access granularity. DecoupleVS exposes a configurable parameter β , the chunk-metadata overhead ratio (chunk metadata size / total vector data size). Let V be the full-precision vector size and $\alpha \in (0, 1]$ be the compression ratio (compressed / uncompressed size; smaller α means greater savings). The expected number of 4 KiB blocks per chunk is $\frac{\alpha \times C}{4096}$, giving a per-chunk metadata size of $4 \times (\frac{\alpha \times C}{4096} + 3) + V$ bytes. This yields $\beta = \frac{V+12}{C} + \frac{\alpha}{1024}$. Given a user-configured β and known α and V , DecoupleVS solves for the required C . When α is unknown, setting $\alpha = 1$ gives a conservative upper bound (i.e., no compression savings).

The sparse in-memory index size is determined by the

number of blocks in the compressed auxiliary index. Specifically, for a neighbor list of degree R and a maximum vector ID N , the worst-case encoded size is at most $2R + R \cdot \lceil \log_2 \frac{N}{R} \rceil$ bits [30]. With N such lists across 4 KiB blocks, the worst-case block count is $\lceil \frac{N(2R + R \lceil \log_2 (N/R) \rceil)}{4096 \times 8} \rceil$. The sparse in-memory index uses 4 bytes per entry, totaling $\lceil \frac{N(2R + R \lceil \log_2 (N/R) \rceil)}{8192} \rceil$ bytes. For instance, with $R = 96$ and $N = 10^8$, the worst-case sparse in-memory index size is about 24.6 MiB. This is consistent with the 19.6 MiB measured on SIFT100M (§3.5), confirming that practical encoding stays well below the bound. Additionally, the per-segment frequency tables have fixed, workload-independent sizes that are small in practice (§4).

3.4 Latency-aware Vector Search

Since the decoupled organization causes search operations to issue separate I/Os to vector data and auxiliary index meta-data, DecoupleVS employs two complementary strategies to preserve graph traversal performance over compressed decoupled storage: (i) *differentiated I/O paths* that keep latency-critical index access on the fast path, and (ii) *adaptive vector prefetching* that avoids unnecessary vector I/Os.

Differentiated I/O paths. DecoupleVS pipelines I/Os and computations (including decompression and distance calculations), as in PipeANN [13], to maximize CPU and I/O utilizations. However, as shown in §2.2, I/O wait time still dominates even with pipelining. The root asymmetry is that graph traversal throughput is limited by neighbor-list access latency, whereas vector data access affects only the recall accuracy of the final re-ranking. Thus, DecoupleVS removes vector data I/Os from the critical search path entirely: during graph traversal, it reads only the compressed auxiliary index to identify candidate vector IDs, caching hot compressed neighbor lists in memory under the least-recently-used (LRU) policy; it separately prefetches the necessary vector data only for final re-ranking.

Since compressed neighbor lists are variable-size, DecoupleVS allocates fixed-size LRU cache entries sized to the Elias-Fano worst-case bound (§3.3): $2R + R \cdot \lceil \log_2 \frac{N}{R} \rceil$ bits per list [30], versus $32(R + 1)$ bits for an uncompressed list (a vertex and its R neighbors). At $R = 128$ and $N = 10^9$ (§4.1), this gives 2,430 bits versus 3,072 bits, achieving at least 20.9% space reduction. By using fixed-size entries, DecoupleVS reserves sufficient space for any compressed neighbor list, while eliminating variable-size allocation overhead.

Adaptive vector prefetching. DecoupleVS prefetches the most promising full-precision vectors during graph traversal and terminates re-ranking early once the result set stabilizes:

- *Phase 1 (Vector prefetching):* DecoupleVS maintains a max-heap of size $K + B$ to track the top candidates by PQ-encoded distance during traversal, where K is the result set size and B is the re-ranking batch size (also used as the prefetch stability threshold). Once the heap is full and B

consecutive candidates are explored without displacing any heap entry, the search is considered stable; using B for both the stability threshold and the re-ranking batch size ensures the prefetch trigger aligns with the re-ranking granularity. DecoupleVS then issues prefetch I/Os for the top K candidates using spare I/O bandwidth, computed as the beam width W [17] minus the number of in-flight traversal I/Os, to avoid contention with ongoing traversal.

- *Phase 2 (Adaptive re-ranking termination):* DecoupleVS immediately begins re-ranking the prefetched vectors while concurrently issuing I/Os for the next batch. For each batch, it computes a *benefit ratio*, the fraction of candidates that displace an entry in the final top- K heap. It terminates re-ranking if the ratio falls below a predefined threshold (default: 0.01), ensuring that I/Os are issued only while the result set continues to improve.

3.5 Decoupled Vector Updates

DecoupleVS preserves storage savings when vectors are inserted or deleted under compressed decoupled storage. It handles vector insertions and deletions through a decoupled update strategy that exploits the asymmetric update requirements of the two storage components: auxiliary index meta-data encodes a highly interconnected graph structure that requires periodic global repair to preserve search accuracy; in contrast, vector data has no inter-vector dependencies as each vector is accessed by ID and can be written independently, making it a natural fit for log-structured storage. Specifically, DecoupleVS applies *batch merges* to the auxiliary index as in FreshDiskANN [34], while writing vector data via segment-level appends (§3.3). It defers space reclamation to background garbage collection (GC) to eliminate full index rewrites at each merge cycle [34]. Note that the auxiliary index path can be adapted to in-place techniques used by IP-DiskANN [43] and OdinANN [14], as they all operate on the same underlying graph structure, while the vector data path is independent of the auxiliary index and naturally benefits from log-structured writes.

Vector insertions. DecoupleVS inserts new vectors into the in-memory Vamana index [17]. When the in-memory index reaches a capacity threshold, it computes neighbor deltas for all valid vectors and applies them to the on-disk auxiliary index in the background. Concurrently, it appends new vectors to the tail of the active mutable segment, and maintains an ID-to-location mapping within each segment group for subsequent lookups and GC.

Vector deletions. For the auxiliary index, DecoupleVS batches deletions and removes references to deleted vectors from their neighbors' adjacency lists to preserve graph connectivity. For vector data, DecoupleVS marks deleted vectors stale and reclaims their space asynchronously by GC. GC is triggered in the background when buffered updates are flushed to disk, to limit interference with foreground operations. It selects segments greedily by *garbage ratio* (i.e.,

fraction of stale vectors) to maximize reclaimed space per I/O. For each selected segment, DecoupleVS copies all valid vectors into a new segment, atomically updates in-memory metadata to redirect lookups to the new locations. It also compresses and seals filled segments during compaction in-line (§3.3). The original stale segments are not released until the index switch completes, since they still serve in-flight foreground queries against the old layout.

Consistency model. DecoupleVS follows a *batch-visible* update model similar to FreshDiskANN [34]. When the current batch of updates is buffered in memory, search queries run against the previous on-disk index and segments. This ensures that newly deleted vectors are never returned, even before their on-disk references are removed. Once the auxiliary-index merge and the subsequent GC cycle complete in the background, DecoupleVS waits for all in-flight queries on the old state to finalize. It then reloads the rebuilt index and the new segment layout, and resumes serving subsequent queries against the new state. The replaced stale segments are safely released only after this switch, without exposing inconsistent vector/index states to concurrent queries.

4 Evaluation

We implement DecoupleVS in C++ based on the PipeANN codebase [13]. PipeANN extends DiskANN [17] by employing `io_uring` as the I/O engine with asynchronous I/O requests to reduce I/O wait time; its codebase comprises 73.8 K LoC. We extend PipeANN with 19.1 K LoC to support vector storage decoupling, lossless compression, and efficient operations on the compressed auxiliary index. For Huffman coding, we adopt a high-performance implementation [45] that exploits CPU out-of-order execution to accelerate compression and decompression. For Elias-Fano encoding, we build on the Succinct library [11]. We parallelize compression and decompression for the auxiliary index to boost throughput.

We evaluate DecoupleVS with the following questions, centered on whether component-aware compression can reduce storage footprint without compromising the performance of disk-resident graph ANNS systems:

- How do DecoupleVS’s individual design components contribute to its search performance? (§4.2)
- How does DecoupleVS perform in terms of storage savings, search performance, and update performance? Can it scale to billion-scale datasets? (§4.3)
- Does DecoupleVS’s tailored compression provide a better storage-performance trade-off for disk-resident graph ANNS than general-purpose compression schemes? (§4.4)

4.1 Methodology

Testbed. We conduct experiments on a physical server equipped with a 32-core 2.30 GHz Intel Xeon Platinum 8336C CPU with 64 hyper-threads, 128 GiB DDR4 memory, and a SOLIDIGM SSDPF2KX038T1 3.84 TiB NVMe SSD, running

Table 2. Datasets used in evaluation.

Source	#Vectors	Type	#Dims	Size	#Queries
DecoupleVS100M	109 M	FP32	128	53 GiB	10 K
SIFT100M [20]	100 M	uint8	128	12 GiB	10 K
SPACEV100M [24]	100 M	int8	100	9.4 GiB	29.3 K
SIFT1B [20]	1 B	uint8	128	120 GiB	10 K
SPACEV1B [24]	1.4 B	int8	100	131 GiB	29.3 K

Debian 9.13 with Linux kernel 5.4.56.

Datasets. Table 2 shows the datasets. DecoupleVS100M is a proprietary dataset of 109 M vectors dumped from a production-scale partition of a commercial search engine that indexes hundreds of billions of vectors. SIFT100M, SPACEV100M, SIFT1B, and SPACEV1B are public datasets; SIFT100M and SPACEV100M are the first 100 M vectors of SIFT1B and SPACEV1B, respectively. SIFT100M and SIFT1B are derived from image descriptors; SPACEV100M and SPACEV1B are derived from web-search embeddings from Microsoft Bing.

Baselines. We compare DecoupleVS against different disk-resident ANNS systems using their open-source prototypes.

- *DiskANN* [17]: a popular graph-based ANNS system employing best-first search with PQ-compressed vectors cached in memory.
- *PipeANN* [13]: an extension of DiskANN that overlaps disk I/Os and distance computations during graph traversal to reduce search latency.
- *FreshDiskANN* [34]: an extension of DiskANN supporting streaming updates via a buffered update strategy.
- *OdinANN* [14]: an extension of PipeANN supporting in-place vector updates to reduce latency and memory usage.
- *SPANN* [2]: an on-disk cluster-based ANNS system that partitions vectors into clusters, connects centroids using an in-memory graph [37], and stores per-centroid posting lists on disk to reduce search I/Os.

We build the graph indexes for PipeANN and DecoupleVS using DiskANN’s index-construction algorithm. We then decouple the storage and apply component-specific compression for DecoupleVS atop the resulting DiskANN index. For 100 M-scale datasets, index construction takes roughly 10 hours, whereas DecoupleVS’s compression and layout transformation complete in about 5 minutes, which is negligible compared to index construction.

Default settings. For DecoupleVS100M, SIFT100M, and SPACEV100M, the graph index uses $R = 96$ outgoing edges per node and a candidate list size of $L_b = 100$. For SIFT1B and SPACEV1B, we increase to $R = 128$ edges and $L_b = 200$ to maintain high recall. For SPANN, we use its default configurations. We set the beam width $W = 4$ for DiskANN, PipeANN, and DecoupleVS, following DiskANN’s default [17]. For fair comparison, we apply the same dynamic cache management (§3.4) to all systems, with a cache capacity of 1 M entries (1% and 0.1% for 100 M and 1 B datasets, respec-

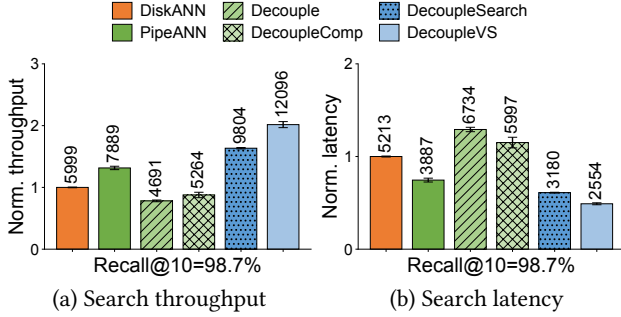


Figure 5. Exp#1 (Effectiveness of each component). Results are normalized to DiskANN. Numbers above bars are search throughput (QPS) and average search latency (ms).

tively). For DecoupleVS, we set the uncompressed chunk size $C = 4$ MiB, the uncompressed segment size to 512 MiB, and the re-ranking batch size $B = 10$. We measure search accuracy by *recall@10*, the fraction of queries whose true nearest neighbors appear in the top-10 results ($K = 10$).

4.2 Microbenchmarks

Exp#1 (Effectiveness of each component). We evaluate the contribution of DecoupleVS’s individual components to search performance by comparing throughput (queries-per-second (QPS)) and average latency (ms) across six configurations: (i) DiskANN, (ii) PipeANN (DiskANN with pipelined I/O), (iii) Decouple (i.e., PipeANN with the decoupled storage layout), (iv) DecoupleComp (i.e., Decouple with tailored lossless compression and hierarchical storage layouts, §3.3), (v) DecoupleSearch (Decouple with latency-aware vector search, §3.4, but compression disabled), and (vi) DecoupleVS. Figure 5 shows throughput and average latency on SIFT100M, with all configurations targeting *recall@10*=98.7%. Overall, DecoupleVS achieves the best performance, with a 2.02× throughput gain and 51.0% latency reduction over DiskANN. We explain these gains by tracing through the configuration stack. PipeANN improves throughput by 31.5% over DiskANN via its I/O-overlapping strategy. Adding the decoupled layout, Decouple degrades throughput by 21.8% and 40.5% over DiskANN and PipeANN, respectively, due to increased I/O overhead from storage separation. DecoupleComp improves 12.2% throughput over Decouple by compressing the auxiliary index, allowing more metadata to reside in the LRU cache and reducing disk I/Os. DecoupleSearch, which adds latency-aware vector search without compression, achieves an 86.3% throughput gain over Decouple, as adaptive vector prefetching removes full-precision vector I/Os from the critical search path and reduces latency by 39.0% over DiskANN. Finally, DecoupleVS enables both compression and latency-aware search, yielding a further 23.4% throughput gain over DecoupleSearch by keeping more compressed index metadata in cache.

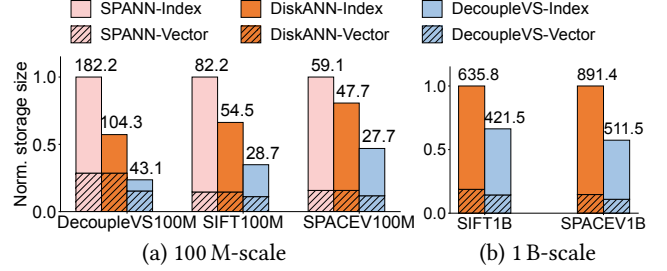


Figure 6. Exp#2 (Storage savings). Sizes are normalized to SPANN for 100 M-scale datasets and to DiskANN for 1 B-scale datasets. Numbers above bars indicate absolute storage sizes (GiB).

4.3 Macrobenchmarks

We study DecoupleVS’s storage and performance efficiencies. DecoupleVS achieves storage savings and search performance improvements through two independent mechanisms, making them *complementary* rather than a trade-off. Its component-aware lossless compression preserves data fidelity, with negligible decompression overhead (§4.4). Its search performance improvements arise independently from latency-aware vector search (§3.4), which removes full-precision vector I/Os from the graph traversal critical path and reduces read amplification regardless of whether compression is used. Thus, DecoupleVS consistently achieves lower storage overhead and higher search performance at similar recall across our evaluated workloads.

Exp#2 (Storage savings). We compare the storage sizes of DecoupleVS against DiskANN and SPANN. We omit PipeANN, which shares DiskANN’s on-disk storage layout and hence identical storage footprint, and omit SPANN from billion-scale comparisons due to excessive memory usage during auxiliary index construction [13, 25].

Figure 6(a) shows that DecoupleVS achieves significant storage savings on 100 M-scale datasets: 76.4%, 65.1%, and 53.1% over SPANN, and 58.7%, 47.4%, and 41.9% over DiskANN, on DecoupleVS100M, SIFT100M, and SPACEV100M, respectively. SPANN’s largest footprint stems from replicating each vector up to 8× across clusters [2]. We further examine the per-component storage breakdown for DiskANN and DecoupleVS. For vector data, DecoupleVS reduces storage by 46.4%, 23.8%, and 25.2% over raw vector sizes on DecoupleVS100M, SIFT100M, and SPACEV100M, respectively. Applying Huffman coding alone achieves only 38.5% compression on DecoupleVS100M. This shows that XOR-delta encoding, when combined with Huffman coding, yields additional savings (§3.2). On SIFT100M and SPACEV100M, however, delta encoding provides no benefit as 8-bit quantization has already increased entropy, leaving limited headroom for further lossless compression. DecoupleVS nonetheless achieves notable reductions on these datasets through Huffman coding alone.

Storage reductions in auxiliary index metadata stem from two sources: (i) DiskANN’s page-aligned fixed-size entries

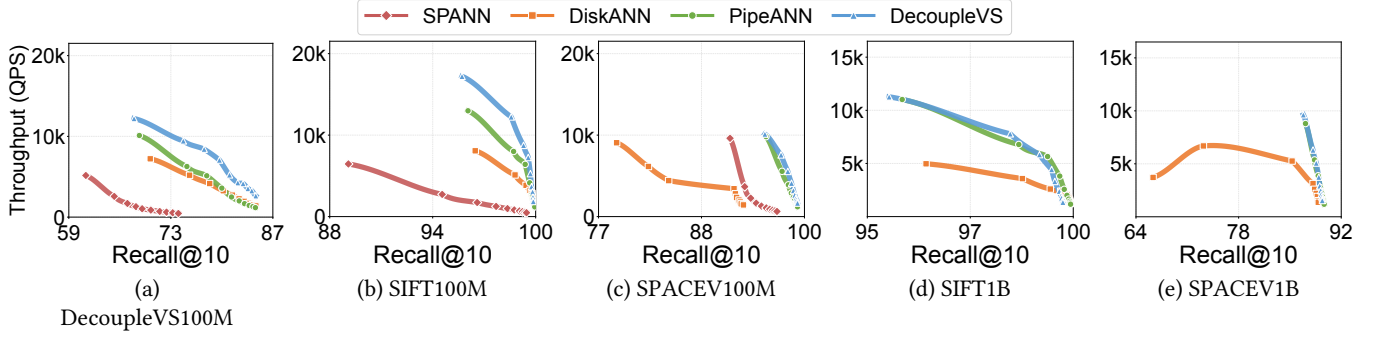


Figure 7. Exp#3 (Search throughput). Throughput (QPS) is plotted against Recall@10 (%); higher is better. Points from left to right in each curve correspond to increasing candidate list sizes, which generally lead to higher recall.

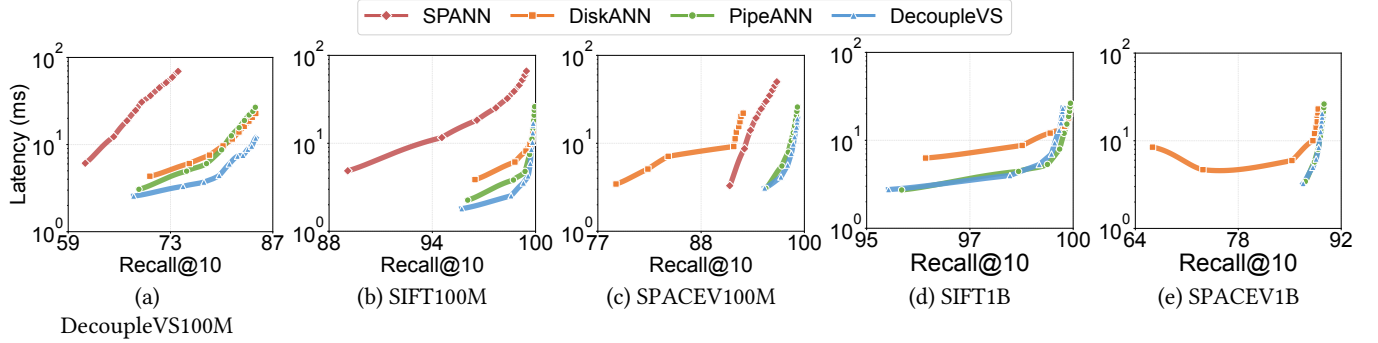


Figure 8. Exp#4 (Search latency). Average latency (ms) is plotted against Recall@10 (%); lower is better.

incur internal fragmentation when vector and metadata sizes do not fit within 4 KiB boundaries. Decoupling mitigates this fragmentation, reducing the auxiliary index size by 19.5%, 14.1%, and 5.92% on DecoupleVS100M, SIFT100M, and SPACEV100M, respectively, compared to DiskANN. (ii) Elias-Fano encoding further reduces the auxiliary index size by 51.5%, 40.0%, and 39.9%, respectively. Together, these optimizations reduce the auxiliary index size by 71.0%, 54.1%, and 45.8% compared to DiskANN on the three datasets.

Figure 6(b) confirms that these gains persist at billion scale: DecoupleVS reduces total storage by 33.7% on SIFT1B and 42.6% on SPACEV1B compared to DiskANN.

Exp#3 (Search throughput). We evaluate search throughput (QPS) of DecoupleVS against SPANN, DiskANN, and PipeANN, varying the search candidate list size L_s from 50 to 500 to obtain 10 accuracy levels for DiskANN and PipeANN (graph-based), and adjusting SPANN’s internal result count within the same range. We use 64 concurrent search threads to maximize throughput.

Figures 7(a)-7(c) show search throughput on the three 100 M-scale datasets. DecoupleVS achieves the highest throughput across all baselines at every accuracy level. On DecoupleVS100M at recall@10 = $83.1\% \pm 0.1\%$, DecoupleVS achieves 1.88 $\times$ and 2.13 $\times$ throughput gains over DiskANN and PipeANN, respectively (Figure 7(a)). On SIFT100M at recall@10 = $98.8\% \pm 0.1\%$, DecoupleVS achieves 2.39 $\times$ and 1.53 $\times$ throughput gains, respectively (Figure 7(b)). These improvements stem from higher cache space utilization and the

differentiated I/O strategy that removes vector fetching from the critical search path (§3.4). Graph-based systems generally outperform SPANN, except DiskANN on SPACEV100M, where SPANN performs slightly better at high accuracy. Both PipeANN and DecoupleVS surpass SPANN’s throughput and peak accuracy: they benefit from PipeANN’s enlarged search space and overlapped I/O execution, which together yield a better accuracy-throughput frontier.

Figures 7(d)-7(e) show that DecoupleVS maintains its advantage on billion-scale datasets: it achieves 2.17 $\times$ and 1.15 $\times$ throughput gains over DiskANN and PipeANN on SIFT1B at recall@10 = $98.7\% \pm 0.1\%$, respectively (Figure 7(d)), and 2.0 $\times$ and 1.17 $\times$ gains on SPACEV1B at recall@10 = $88.8\% \pm 0.1\%$, respectively (Figure 7(e)).

Exp#4 (Search latency). We evaluate the average search latency (ms), following the same setting in Exp#3. Figure 8 shows that the latency results are consistent with the throughput findings. On DecoupleVS100M at recall@10 = $83.1\% \pm 0.1\%$, DecoupleVS reduces latency by 47.2% over DiskANN and 52.9% over PipeANN (Figure 8(a)). On SIFT1B at recall@10 = $98.7\% \pm 0.1\%$, DecoupleVS reduces latency by 54.6% over DiskANN and 11.0% over PipeANN (Figure 8(d)). In Appendix (see the supplementary file), we show that DecoupleVS reduces P99 tail latency by 51.9% over both DiskANN and PipeANN on DecoupleVS100M at recall@10 = $83.0\% \pm 0.1\%$, confirming that its latency advantage holds at the tail.

Exp#5 (Concurrent search and update perfor-

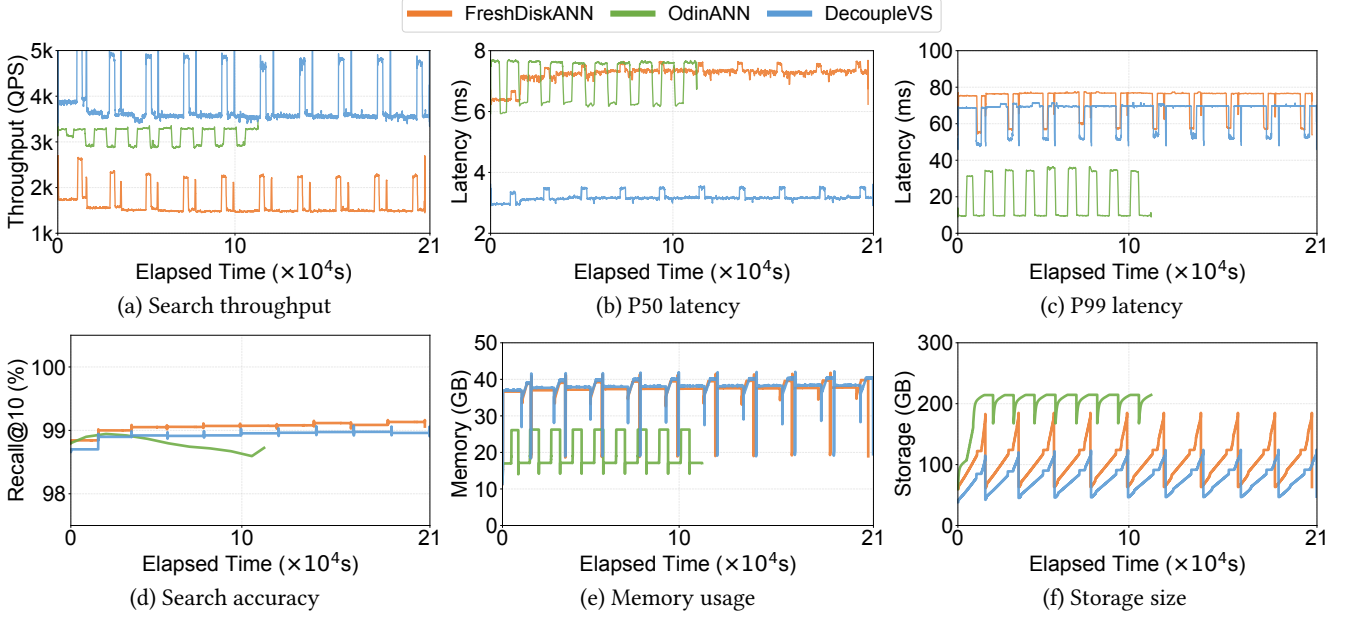


Figure 9. Exp#5 (Concurrent search and updates). Performance is measured on SIFT100M.

mance). We compare DecoupleVS against FreshDiskANN [34](buffered updates) and OdinANN [14] (in-place updates). We use the best-first search for FreshDiskANN and PipeSearch [13] for OdinANN. All systems are indexed on SIFT100M. As in [14, 34], we simulate a streaming update workload by replacing 50% of vectors over 10 iterations, each merging 5% deletions and 5% insertions. We launch 32 concurrent search threads and 32 update threads, and measure throughput, P50 and P99 latencies, recall, memory usage, and storage size throughout.

Figures 9(a)-9(b) show that DecoupleVS consistently achieves higher throughput and lower P50 latency than both baselines, with average throughput gains of 2.31 $\times$ and 1.21 $\times$ over FreshDiskANN and OdinANN, and average P50 latency reductions of 55.5% and 55.4%, respectively. These gains stem from DecoupleVS’s latency-aware search strategy (§3.4), which mitigates I/O interference from concurrent updates. OdinANN, however, achieves the lowest P99 latency, reducing it by 77.5% and 75.0% compared to FreshDiskANN and DecoupleVS, respectively (Figure 9(c)), as its delta neighbor pruning substantially reduces per-update computation, alleviating tail latency. Delta neighbor pruning is orthogonal to DecoupleVS’s design and can be incorporated to further reduce P99 latency. In terms of recall, DecoupleVS matches FreshDiskANN within 0.13% on average, while OdinANN incurs a 0.26% average loss (Figure 9(d)).

Figures 9(e)-9(f) show space efficiency during updates. DecoupleVS’s peak memory usage is only 0.98% higher than FreshDiskANN’s, attributed to: (i) in-memory compression metadata (28 MiB on SIFT100M), (ii) compression and decompression buffers during updates, and (iii) larger per-batch entry counts under the compressed layout. For SIFT100M, the

compression metadata totals 28.5 MiB (including 19.6 MiB for the sparse in-memory index, 8.84 MiB for chunk metadata, and 30 KiB for Huffman codebooks), accounting for only 0.095% of storage size. OdinANN reduces memory usage by 37.2% over FreshDiskANN and 37.8% over DecoupleVS by avoiding in-memory index buffering. At the storage layer, DecoupleVS reduces footprint by 26.9% over FreshDiskANN and 61.8% over OdinANN; OdinANN’s largest footprint is due to the additional on-disk structures required for in-place updates. The stable storage size across merge iterations confirms that GC successfully reclaims stale vector space.

In summary, DecoupleVS achieves the best search performance during concurrent updates with storage overhead comparable to FreshDiskANN. These results reflect an inherent trade-off between update strategies: buffered updates (in FreshDiskANN and DecoupleVS) preserve storage efficiency and search performance at the cost of higher memory usage, while in-place updates (in OdinANN) reduce memory at the cost of a larger storage footprint. Notably, both strategies are orthogonal to DecoupleVS’s decoupled storage layout and can be combined with it to achieve high space efficiency.

4.4 System Analysis

We analyze SIFT100M to provide detailed breakdowns of search and update performance. We also study the impact of compression algorithms using the 100 M-scale datasets.

Exp#6 (Search performance breakdown). We decompose the per-search-query resource usage of DiskANN, PipeANN, and DecoupleVS at $L_s = 100$ into I/O and CPU components: the I/O usage component include: (i) graph cache hit count, (ii) graph traversal disk I/O count, (iii) vector fetch disk I/O count, and (iv) total disk I/O time; the CPU metrics include: (i)

Table 3. Exp#6 (Search performance breakdown). We show the average per-search-query breakdown latencies (ms), including 95% confidence intervals.

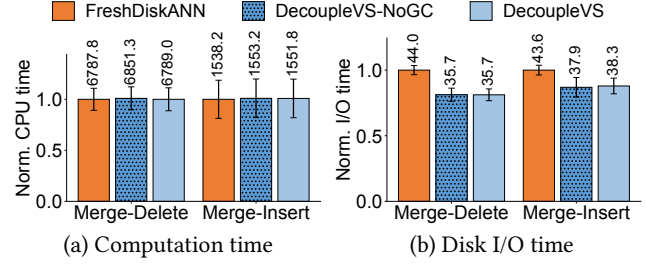
Steps	DiskANN	PipeANN	DecoupleVS
Total time	5.67±0.60	3.97±0.04	2.48±0.14
I/O usage			
Graph cache hits	66.3±0.01	44.1±0.02	97.3±0.08
Graph I/Os	64.5±0.01	67.9±0.03	10.9±0.09
Vector I/Os	-	-	33.2±0.03
Total I/O time	2.33±0.04	2.62±0.14	1.34±0.07
CPU usage			
Graph decompression	-	-	0.031±0.008
PQ calculation	0.64±0.02	0.60±0.07	0.44±0.04
Vector decompression	-	-	0.070±0.005
Rerank calculation	0.023±0.005	0.012±0.005	0.0028±0.0001

* DiskANN and PipeANN co-locate vector data with auxiliary index meta-data, so no separate vector I/O step is counted.

graph decompression time, (ii) PQ distance calculation time, (iii) vector decompression time, and (iv) re-ranking distance calculation time. Each metric is averaged across all queries. As 64 search threads run concurrently, the wall-clock time per query exceeds the sum of individual step times due to inter-thread contention.

Table 3 shows the breakdown. For I/O usage, DecoupleVS is highly cache-efficient, increasing graph cache hit count by 46.8% over DiskANN and 120.6% over PipeANN. PipeANN has the lowest cache hit count since its enlarged search space evicts entries more aggressively, resulting in the highest total disk I/Os (67.9 versus DiskANN’s 64.5 and DecoupleVS’s 44.1) and the highest total disk I/O time. PipeANN partially recovers by overlapping I/O with computation, yielding lower overall latency than DiskANN despite more I/Os. For CPU usage, DecoupleVS’s decompression overhead, covering both neighbor lists and full-precision vectors, accounts for only 4.1% of average query latency, confirming that compression does not impede search. DiskANN exhibits the highest CPU computation time because its blocking I/O model causes frequent context switches across threads, thereby increasing the number of vectors visited. DecoupleVS’s PQ calculation time is higher than PipeANN’s due to higher CPU utilization from reduced I/O wait time. In summary, disk I/O is the dominant cost in all systems; DecoupleVS reduces I/O time by 42.5% over DiskANN and 48.9% over PipeANN, confirming its effectiveness in reducing read amplification.

Exp#7 (Update performance breakdown). We analyze the two most time-consuming update operations in DecoupleVS: (i) Merge-Delete (merging deletion markers into the on-disk index) and (ii) Merge-Insert (merging newly inserted points into the on-disk index). Each component is decomposed into computation and disk I/O time, and the results are averaged over 10 iterations, with 95% confidence intervals based on the Student’s t-distribution. We compare

**Figure 10.** Exp#7 (Update performance breakdown). Average computation and disk I/O time per update operation are normalized to DiskANN. Numbers above bars indicate absolute time in seconds.

FreshDiskANN, DecoupleVS, and DecoupleVS-NoGC (a DecoupleVS variant with GC disabled to isolate the GC impact on update performance).

Figure 10 shows the results normalized to FreshDiskANN. The computation time is nearly identical across all three schemes: DecoupleVS incurs only 0.018% and 0.88% increases over FreshDiskANN for Merge-Delete and Merge-Insert, respectively, attributed to compression and decompression. DecoupleVS reduces disk I/O time by 18.9% for Merge-Delete and 12.2% for Merge-Insert over FreshDiskANN, confirming that DecoupleVS’s decoupled layout reduces write amplification. DecoupleVS and DecoupleVS-NoGC differ by at most 0.9% in computation time and 1.1% in disk I/O time, both within the margin of error, indicating that GC imposes negligible overhead on foreground update operations.

Exp#8 (Compression algorithm analysis). We compare DecoupleVS against two general-purpose lossless compression techniques, ZSTD and Huffman, using the Zstandard library [4], which implements both algorithms. Figure 11(a) shows the storage reduction on the auxiliary index metadata as a function of neighbor list size R . DecoupleVS consistently achieves the largest reduction, as Elias-Fano encoding is specifically designed for monotone integer sequences. At $R = 96$, DecoupleVS reduces index size by 48.6%, versus 31.0% for both ZSTD and Huffman. The advantage of DecoupleVS grows with a larger R , since longer sorted integer lists become increasingly compressible under Elias-Fano’s two-level representation. Figure 11(b) shows compression performance on different 100 M-scale datasets. ZSTD achieves 51.1% reduction versus DecoupleVS’s 47.0%, as ZSTD compresses at least 128 KiB at a time, exploiting cross-vector patterns at a wider context. However, this requires decompressing an entire 128 KiB block to retrieve a single vector, making it unsuitable for the random per-vector access pattern of ANNS. DecoupleVS compresses each vector independently, preserving fine-grained random access at the cost of a modest compression gap. Compared to Huffman alone, DecoupleVS achieves 13.8% additional reduction on DecoupleVS100M via XOR-delta encoding; on SIFT100M and SPACEV100M, where 8-bit quantization has already maximized entropy, both methods achieve the same compression ratio, as reported in Exp#2.

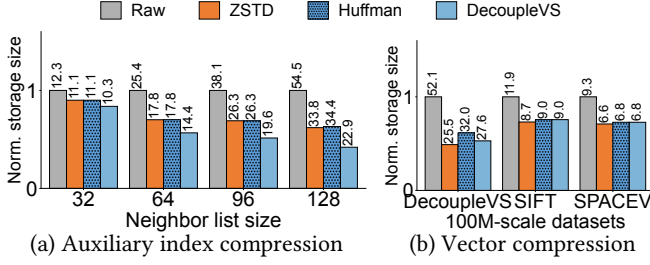


Figure 11. Exp#8 (Compression algorithm analysis). Sizes are normalized to the raw size, and numbers atop the bars indicate absolute storage sizes in GiB.

4.5 Discussion on Parameter Settings

DecoupleVS exposes five configurable parameters: (i) chunk size C , (ii) segment size, (iii) cache size, (iv) re-ranking batch size B , and (v) benefit ratio. Our default values (§4.1) are based on the following principled rationale. We set $C = 4$ MiB to keep per-chunk metadata overhead β (§3.3) within a practical bound across all evaluated datasets (i.e., within 0.1% of the dataset size). We adopt a segment size of 512 MiB following conventions in popular vector databases [46]. The cache ratio is capped at 1% of the dataset size, a practical memory footprint that avoids excessive DRAM pressure. We set $B = 10$ to match the result set size $K = 10$, so that each re-ranking batch covers exactly one full result’s worth of candidates, balancing I/O cost and re-ranking coverage; we also use the same B for the prefetch stability threshold to align prefetching and re-ranking in latency-aware vector search. The benefit ratio is set to 0.01, reflecting the empirical observation that result-set changes become negligible below this threshold; setting it lower risks unnecessary I/Os while setting it higher risks premature termination.

5 Related Work

Disk-resident ANNS systems. ANNS indexes are broadly categorized into graph-based and cluster-based. Graph-based indexes are pioneered by DiskANN [17], which constructs a disk-resident index graph for efficient similarity search; FreshDiskANN [34] extends DiskANN to support efficient buffered updates for streaming workloads. Starling [39] co-locates neighboring graph vertices on the same disk pages to reduce random I/Os. PipeANN [13] overlaps I/Os and computations during graph traversal via pipelined execution. OdinANN [14] supports in-place updates and improves update performance using write-back caching and delta neighbor pruning. Cluster-based indexes are pioneered by SPANN [2], which partitions vectors into clusters and attaches on-disk posting lists to each centroid for efficient search. SPFresh [44] extends SPANN to support efficient in-place streaming updates. Recent extensions such as SmartANNS [35] and FusionANN [36] accelerate search by offloading distance computations to SmartSSDs or GPUs. In contrast, DecoupleVS focuses on improving space efficiency while maintaining

high search and update performance.

Concurrent to our work, DGAI [22] and VeloANN [48] both redesign the on-disk storage layout to improve I/O efficiency during search and updates. DGAI focuses on optimizing update efficiency and query performance through a similarity-aware dynamic layout and hierarchical PQ, without addressing storage efficiency; its decoupled design incurs up to 27.8% additional storage overhead. VeloANN introduces a lossy compressed vector layout with affinity-based record co-placement to maximize search throughput; however, its lossy compressed layout does not preserve data fidelity, and it targets static workloads without addressing the impact of per-record updates on search performance. DecoupleVS targets component-aware lossless compression for persistent vector storage to preserve data fidelity while maintaining high search and update performance.

Low-storage ANNS. Disk-resident ANNS systems [17, 36] often cache lossy-compressed vectors in memory. Product quantization (PQ) [18] is the predominant lossy vector compression technique; it represents high-dimensional vectors with compact codes by quantizing sub-dimensions into codebooks. To improve quantization accuracy, Optimized-PQ [10] reduces inter-dimensional correlation via orthogonal rotation preprocessing on the original vectors, while Ra-BitQ [9] adopts an integrated design of codebook construction and distance estimation. However, they inevitably incur quantization errors in search. To meet high recall, existing disk-resident ANNS systems [17, 36] still store full-precision vectors on disk for a final re-ranking step. LEANN [40] eliminates full-precision vector storage by recomputing embeddings on the fly during search for low storage overhead. However, it incurs significant recomputation latency during search. DecoupleVS applies component-aware lossless compression to persistent vector storage for space savings with negligible decompression overhead.

General-purpose lossless compression. Lossless compression reduces data volume while ensuring perfect reconstruction. General-purpose lossless compression methods, such as entropy coders (e.g., Huffman [19], ANS [6]) and dictionary coders (e.g., LZ77 [32] and LZ4 [3]) are widely adopted, but overlook data-specific characteristics. Domain-specific techniques achieve higher compression savings by exploiting inherent data properties, such as AI model weights [16], LLM storage [41], and sorted inverted lists [30, 38]. Following this principle, DecoupleVS designs tailored lossless compression for both vector data and auxiliary index metadata, leveraging their distinct data locality patterns to maximize storage savings without degrading performance.

6 Conclusion

DecoupleVS addresses the storage compression problem in disk-resident graph ANNS systems through component-aware compression to data-index decoupled storage. With

carefully designed lossless compression techniques, data layouts, and search/update paths, DecoupleVS addresses the space, read, and write inefficiencies of existing co-located designs while preserving search and update performance. Evaluation on real-world public and proprietary billion-scale datasets shows that DecoupleVS substantially reduces storage space while delivering improved or competitive search and update performance compared to state-of-the-art disk-resident graph ANNS systems.

References

- [1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Proc. of NeurIPS*, 2020.
- [2] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. SPANN: Highly-efficient billion-scale approximate nearest neighborhood search. *Proc. of NeurIPS*, 2021.
- [3] Yann Collet. LZ4. <https://github.com/lz4/lz4>, 2011.
- [4] Yann Collet and Chip Turner. Zstandard. <https://github.com/facebook/zstd>, 2015.
- [5] Paul Covington, Jay Adams, and Emre Sargin. Deep neural networks for YouTube recommendations. In *Proc. of ACM RecSys*, 2016.
- [6] Jarek Duda. Asymmetric numeral systems: Entropy coding combining speed of Huffman coding with compression rate of arithmetic coding. *arXiv preprint arXiv:1311.2540*, 2013.
- [7] Peter Elias. Efficient storage and retrieval by content and address of static files. *Journal of the ACM (JACM)*, 21(2):246–260, 1974.
- [8] Robert Mario Fano. *On the number of bits required to implement an associative memory*. Massachusetts Institute of Technology, Project MAC, 1971.
- [9] Jianyang Gao and Cheng Long. RabbitQ: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search. *Proc. of ACM SIGMOD*, 2(3):1–27, 2024.
- [10] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. Optimized product quantization. *IEEE Trans. on Pattern Analysis and Machine Intelligence*, 36(4):744–755, 2013.
- [11] Giuseppe Ottaviano. Succinct. <https://github.com/ot/succinct>, 2017.
- [12] Mihajlo Grbovic and Haibin Cheng. Real-time personalization using embeddings for search ranking at airbnb. In *Proc. of the 24th ACM SIGKDD*, 2018.
- [13] Hao Guo and Youyou Lu. Achieving low-latency graph-based vector search via aligning best-first search algorithm with SSD. In *Proc. of USENIX OSDI*, 2025.
- [14] Hao Guo and Youyou Lu. OdinANN: Direct insert for consistently stable performance in billion-scale graph-based vector search. In *Proc. of USENIX FAST*, 2026.
- [15] Harsha Simhadri. Research talk: Approximate nearest neighbor search systems at scale. <https://youtu.be/BnYNdSIKibQ?t=179>, 2022.
- [16] Moshik Herscovitch, Andrew Wood, Leshem Choshen, Guy Girmonsky, Roy Leibovitz, Or Ozeri, Ilias Ennmouri, Michal Malka, Peter Chin, Swaminathan Sundararaman, et al. ZipNN: Lossless compression for AI models. In *Proc. of IEEE CLOUD*, 2025.
- [17] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. *Proc. of NeurIPS*, 2019.
- [18] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Trans. on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2010.
- [19] Donald E Knuth. Dynamic Huffman coding. *Journal of Algorithms*, 6(2):163–180, 1985.
- [20] Laurent Amsaleg and Hervé Jégou. Datasets for approximate nearest neighbor search. <http://corpus-texmex.irisa.fr/>, 2010.
- [21] Shengwen Liang, Ying Wang, Ziming Yuan, Cheng Liu, Huawei Li, and Xiaowei Li. VStore: in-storage graph based vector search accelerator. In *Proc. of ACM/IEEE DAC*, 2022.
- [22] Jiahao Lou, Quan Yu, Shufeng Gong, Song Yu, Yanfeng Zhang, and Ge Yu. DGAI: Decoupled on-disk graph-based ann index for efficient updates and queries. *arXiv e-prints*, 2026.
- [23] Yu A Malkov and Dmitry A Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Trans. on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2018.
- [24] Microsoft. SPACEV1B: A billion-scale vector dataset for text descriptors. <https://github.com/microsoft/SPTAG/tree/main/datasets/SPACEV1B>.
- [25] Microsoft. SPTAG issue #416: Segmentation fault when building SIFT1B. <https://github.com/microsoft/SPTAG/issues/416>, 2024.
- [26] Marius Muja and David Lowe. Flann-fast library for approximate nearest neighbors user manual. *Computer Science Department, University of British Columbia, Vancouver, BC, Canada*, 5(6), 2009.
- [27] NeurIPS. BIG ANN-Benchmarks. <https://big-ann-benchmarks.com/neurips21.html>, 2021.
- [28] Wanyi Ning, Jingyu Wang, Qi Qi, Mengde Zhu, Haifeng Sun, Daixuan Cheng, Jianxin Liao, and Ce Zhang. Fm-delta: Lossless compression for storing massive fine-tuned foundation models. *Proc. of NeurIPS*, 2024.
- [29] OpenAI. The ChatGPT Retrieval Plugin lets you easily search and find personal or work documents by asking questions in everyday language. <https://github.com/openai/chatgpt-retrieval-plugin>, 2023.
- [30] Giuseppe Ottaviano and Rossano Venturini. Partitioned elias-fano indexes. In *Proceedings of the 37th International ACM SIGIR Conference on Research & Development in Information Retrieval*, 2014.
- [31] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proc. of the EMNLP*, 2019.
- [32] Suzanne Rigler, William Bishop, and Andrew Kennings. FPGA-based lossless data compression using Huffman and LZ77 algorithms. In *2007 Canadian conference on electrical and computer engineering*, pages 1235–1238. IEEE, 2007.
- [33] Claude E Shannon. A mathematical theory of communication. *The Bell system technical journal*, 27(3):379–423, 1948.
- [34] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. FreshDiskANN: A fast and accurate graph-based ann index for streaming similarity search. *arXiv preprint arXiv:2105.09613*, 2021.
- [35] Bing Tian, Haikun Liu, Zhuohui Duan, Xiaofei Liao, Hai Jin, and Yu Zhang. Scalable billion-point approximate nearest neighbor search using SmartSSDs. In *Proc. of USENIX ATC*, 2024.
- [36] Bing Tian, Haikun Liu, Yuhang Tang, Shihai Xiao, Zhuohui Duan, Xiaofei Liao, Hai Jin, Xuechang Zhang, Junhua Zhu, and Yu Zhang. Towards high-throughput and low-latency billion-scale vector search via CPU/GPU collaborative filtering and re-ranking. In *Proc. of USENIX FAST*, 2025.
- [37] Godfried T Toussaint. The relative neighbourhood graph of a finite planar set. *Pattern recognition*, 12(4):261–268, 1980.
- [38] Jianguo Wang, Chunbin Lin, Ruining He, Moojin Chae, Yannis Papakonstantinou, and Steven Swanson. MILC: Inverted list compression in memory. *Proceedings of the VLDB Endowment*, 10(8):853–864, 2017.
- [39] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu, Rentong Guo, and Charles Xie. Starling: An I/O-efficient disk-resident graph index framework for high-dimensional vector similarity search on data segment. *Proc. of ACM SIGMOD*, 2024.
- [40] Yichuan Wang, Zhifei Li, Shu Liu, Yongji Wu, Ziming Mao, Yilong

- Zhao, Xiao Yan, Zhiying Xu, Yang Zhou, Ion Stoica, Sewon Min, Matei Zaharia, and Joseph E. Gonzalez. LEANN: A low-storage overhead vector index. *arXiv preprint arXiv:2506.08276*, 2025.
- [41] Zirui Wang, Tingfeng Lan, Zhaoyuan Su, Juncheng Yang, and Yue Cheng. ZipLLM: towards efficient LLM storage reduction via tensor deduplication and delta compression. In *Proc. of USENIX NSDI*, 2026.
- [42] Weaviate. Configure replication in Weaviate ANN service. <https://docs.weaviate.io/deploy/configuration/replication>, 2025.
- [43] Haike Xu, Magdalen Dobson Manohar, Philip A Bernstein, Badrish Chandramouli, Richard Wen, and Harsha Vardhan Simhadri. In-place updates of a graph index for streaming approximate nearest neighbor search. *arXiv preprint arXiv:2502.13826*, 2025.
- [44] Yuming Xu, Hengyu Liang, Jin Li, Shuotao Xu, Qi Chen, Qianxi Zhang, Cheng Li, Ziyue Yang, Fan Yang, Yuqing Yang, et al. SPFresh: Incremental in-place update for billion-scale vector search. In *Proc. of ACM SOSR*, 2023.
- [45] Yann Collet. New Generation Entropy coders. <https://github.com/Cyan4973/FiniteStateEntropy>, 2019.
- [46] Ken Zhang and Fendy Feng. Introducing the Milvus Sizing Tool: Calculating and Optimizing Your Milvus Deployment Resources. <https://milvus.io/blog/introducing-the-milvus-sizing-tool-calculating-and-optimizing-your-milvus-deployment-resources.md>, 2025.
- [47] Zili Zhang, Fangyue Liu, Gang Huang, Xuanzhe Liu, and Xin Jin. Fast vector query processing for large datasets beyond GPU memory with reordered pipelining. In *Proc. of USENIX NSDI*, 2024.
- [48] Weichen Zhao, Yuncheng Lu, Yao Tian, Hao Zhang, Jiehui Li, Minghao Zhao, Yakun Li, and Weining Qian. Optimizing ssd-resident graph indexing for high-throughput vector search. *arXiv preprint arXiv:2602.22805*, 2026.

A Appendix

We report additional evaluation results that supplement the findings in the main paper.

Exp#9 (P99 tail latency versus accuracy). We evaluate the P99 tail latency of DecoupleVS against DiskANN and PipeANN on DecoupleVS100M and SIFT100M, using the same experimental settings as Exp#4 (§4.3).

Figure 12 shows the results. On DecoupleVS100M, DecoupleVS consistently reduces P99 latency across all recall levels. At $\text{recall@10} = 83.0\% \pm 0.1\%$, DecoupleVS reduces P99 latency by 51.9% over both DiskANN and PipeANN. At $\text{recall@10} = 85.5\% \pm 0.1\%$, the reductions are 50.1% over DiskANN and 50.9% over PipeANN. These improvements are consistent with the average latency reductions reported in Exp#4, confirming that DecoupleVS’s latency-aware search strategy (§3.4) reduces not only mean latency but also worst-case tail latency by removing full-precision vector fetching from the critical search path.

On SIFT100M, DecoupleVS reduces P99 latency at moderate recall levels: at $\text{recall@10} = 98.7\% \pm 0.1\%$, DecoupleVS

achieves reductions of 34.2% over DiskANN and 18.5% over PipeANN. At very high recall (above 99.8%), all systems must traverse a larger portion of the graph to meet accuracy targets, reducing the opportunity for adaptive prefetching and narrowing the P99 gap across systems.

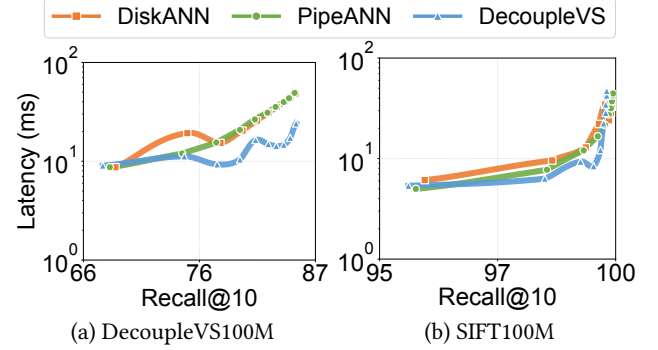


Figure 12. Exp#9 (P99 tail latency versus accuracy). P99 latency (ms) is plotted against Recall@10 (%); lower is better.